

Guía de Aprendizaje: Procesos Huérfanos y Zombies

Malak Sanchez

Monitorías de Sistemas Operativos

25 de marzo de 2026

1. Motivación

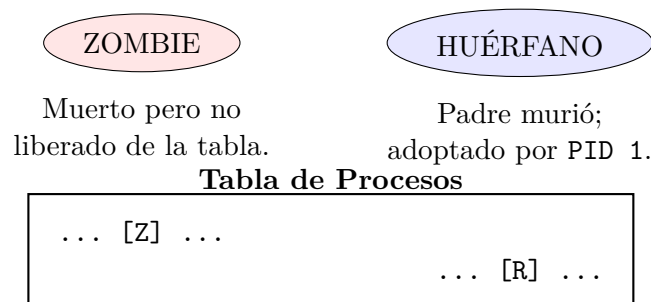
En la vida real, un fallecimiento requiere trámites legales para cerrar las cuentas de una persona. En Linux ocurre algo similar: cuando un proceso termina, no puede desaparecer por completo hasta que su padre reconozca su finalización. Entender estos “cabos sueltos” es vital para evitar fugas de recursos en el sistema.

2. Idea Fundamental

Existen dos situaciones anómalas en la jerarquía:

- **Huérfano:** El padre terminó primero; el hijo sigue vivo y necesita un nuevo tutor.
- **Zombie:** El hijo terminó; ya no consume CPU ni RAM, pero sigue ocupando una entrada en la tabla de procesos porque el padre no ha preguntado por su estado.

3. Visualización de Estados



4. Sintaxis Básica

No hay funciones para “crear” estados, ya que estos estados ocurren por el flujo del programa:

```
exit(status); // Hijo muere -> Se vuelve Zombie hasta un wait()  
// El padre termina sin hacer wait() -> Hijo se vuelve Huerfano
```

5. Ejemplo Mínimo Funcional (Zombie)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     if(fork() == 0){
7         printf("[HIJO] Terminando...\n");
8         exit(0); // El hijo muere
9     }
10    // El padre duerme y NO hace wait()
11    printf("[PADRE] Durmiendo sin esperar al hijo...\n");
12    sleep(10);
13    return 0;
14 }
```

5.1. Explicación Paso a Paso

1. El hijo termina su ejecución inmediatamente.
2. Como el padre está en `sleep`, no llama a `wait()`.
3. Si revisas la terminal con `ps -l`, verás al proceso con estado **Z** o *<defunct>*.
4. Cuando el padre termine (y con él el programa), el Zombie desaparecerá definitivamente.

6. Ejemplo Mínimo Funcional (Huérfano)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main() {
6     if(fork() == 0){
7         printf("[HIJO] Mi padre es: %d\n", getppid());
8         printf("[HIJO] Durmiendo mientras el padre termina...\n");
9         ;
10        sleep(5);
11        printf("[HIJO] Ahora mi padre es: %d (adoptado)\n",
12               getppid());
13        exit(0);
14    }
15    // El padre termina de inmediato
16    printf("[PADRE] Terminando mi ejecucion...\n");
17    exit(0);
18 }
```

6.1. Explicación Paso a Paso

1. El padre termina su ejecución inmediatamente despues de crear el proceso hijo.
2. El hijo ve que su padre ha muerto, por lo que es adoptado por el proceso `init` (PID 1).
3. El hijo termina (y con él el programa), el Zombie desaparecerá definitivamente.

7. Qué ocurre en memoria

Cuando un proceso termina (`exit`), el SO libera sus recursos activos (código, datos, archivos). Sin embargo, reserva una pequeña estructura en la Tabla de Procesos que contiene el PID y el estado de salida. Esto se mantiene hasta que el padre ejecute un `wait()`. Si la tabla se llena de Zombies, el sistema no podrá crear nuevos procesos.

8. Patrones comunes de uso

Evitar Zombies es obligatorio mediante el uso de `wait()` o ignorando señales (revisar capítulo 2) `SIGCHLD`:

```
1 #include <signal.h>
2 signal(SIGCHLD, SIG_IGN); // Una forma de evitar que se formen
   zombies
```

9. Errores Comunes

- **Confusión:** Creer que un Zombie consume memoria RAM (en realidad solo ocupa un espacio ínfimo en la tabla del Kernel).
- **Miedo a los huérfanos:** Los huérfanos no son peligrosos, el SO (`init`) se encarga de ellos eficientemente.

10. Buenas Prácticas

- Un proceso padre responsable siempre debe “cosechar” a sus hijos usando `wait()`.
- Si un programa padre va a vivir mucho tiempo (un servidor), vigilar la creación de procesos hijos para no saturar la tabla.

11. Ejercicios

1. Cree un proceso huérfano: haga que el padre imprima su PID y termine, mientras el hijo duerme y luego imprime su nuevo PPID.
2. Investigue el comando `top` o `ps aux`. ¿Cómo puede identificar gráficamente a un proceso zombie?